

Machine Learning Approaches for Cloud Detection in Multispectral Satellite Imagery

Milena Perez-Gerus

Code: <https://github.com/mimiperezg/satellite-cloud-classification>

April 24, 2026

Abstract

Cloud detection is a critical preprocessing step in satellite image analysis, as cloud cover can obscure surface features and reduce the reliability of remote sensing applications. In this study, we address patch-level cloud classification using the 38-Cloud dataset, reformulating the problem as a binary classification task based on multi-spectral image patches. We evaluate three models: Random Forest, Support Vector Machine (SVM), and a deep convolutional neural network (CNN). For Random Forest and SVM models, we extract statistical features from spectral bands and incorporate domain-informed features such as normalized difference vegetation index (NDVI) and brightness. The CNN operates directly on raw multi-spectral image data, learning spatial and spectral patterns without feature engineering. Results show that feature engineering improves the performance of classical models, with NDVI emerging as a key feature for cloud detection. The SVM achieves strong performance using a nonlinear kernel, while the Random Forest provides interpretable feature importance. The CNN achieves the highest overall accuracy when properly tuned but requires careful regularization to prevent overfitting. These findings highlight a trade-off between interpretability and predictive performance, where classical models offer insight into feature importance, while deep learning models capture more complex patterns for improved accuracy.

1 Introduction

Clouds frequently obscure the Earth’s surface in optical satellite imagery, reducing the reliability of remote sensing observations. Global satellite observations estimate that clouds cover roughly 67% of the Earth’s surface at any given time [3]. Detecting cloud-covered regions is therefore an essential preprocessing step for applications such as land cover classification, wildfire monitoring, agricultural assessment, and air pollution analysis.

Despite its importance, cloud detection remains challenging. Clouds vary significantly in shape, texture, and optical properties, and thin clouds or haze can be difficult to distinguish from underlying land or water features [7]. Additionally, bright surfaces such as snow, ice, or sand can exhibit spectral characteristics similar to clouds, leading to false detections [1].

These challenges motivate the use of machine learning methods that can capture complex spectral and spatial patterns.

Recent work has explored deep learning approaches, particularly convolutional neural networks (CNNs) [6], due to their ability to learn spatial features directly from imagery. However, classical machine learning models such as Random Forests and Support Vector Machines (SVMs) remain widely used due to their interpretability, efficiency, and strong performance on structured features. Comparing these approaches provides insight into the trade-offs between model complexity and performance.

In this study, we investigate patch-level cloud classification using the 38-Cloud dataset, which consists of 384×384 patches derived from Landsat 8 scenes. We formulate a binary classification task based on cloud presence and evaluate three models: Random Forest, Support Vector Machine (SVM), and a deep convolutional neural network (CNN). For Random Forest and SVM models, we use statistical and domain-informed features such as NDVI and brightness, while the CNN operates directly on raw multi-spectral image patches. We analyze model performance with respect to feature engineering and hyperparameter tuning.

2 Experimental Setup

2.1 Data Preprocessing

We used the 38-Cloud dataset [5, 4], which consists of 384×384 pixel image patches derived from 38 Landsat 8 satellite scenes. The dataset contains 8,400 patches, each including four spectral bands corresponding to red, green, blue, and near-infrared (NIR), along with a ground-truth cloud mask. Although the dataset is originally designed for semantic cloud segmentation, we reformulate the task as a patch-level binary image classification problem. An example image patch and its corresponding ground-truth cloud mask are shown in Figure 1.

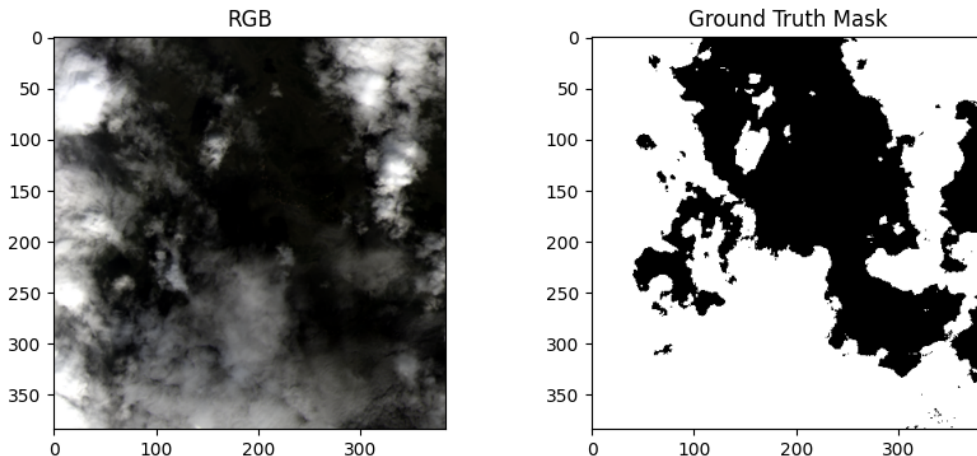


Figure 1: Example satellite image patch (left) and corresponding ground-truth cloud mask (right).

To ensure data quality and relevance for classification, we restrict our analysis to a subset

of informative patches provided by the dataset authors. Due to the geometry of Landsat 8 scenes, empty triangular regions appear at the edges of images, resulting in some extracted patches containing only zero-valued pixels. The dataset authors provide a list of informative patches containing more than 80% nonzero pixels. We use this filter to remove empty or non-informative patches, which would otherwise introduce noise and bias into the learning process. The resulting dataset size is 5,155 patches.

For each patch, we computed the proportion of cloud pixels using the ground-truth mask. Binary labels were assigned based on a cloud coverage threshold of 20%: patches with cloud fraction greater than or equal to the threshold were labeled as *cloud*, while the remaining patches were labeled as *no cloud*. The resulting dataset exhibits moderate class imbalance, with 1,749 no-cloud samples (33.9%) and 3,406 cloud samples (66.1%).

For Random Forest and SVM models, we extracted statistical features from each spectral band, including the mean, standard deviation, minimum, and maximum values. In addition, we engineered two domain-relevant features: normalized difference vegetation index (NDVI) and overall brightness [2]. NDVI was computed as:

$$\text{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red} + \epsilon} \quad (1)$$

Brightness was defined as the average intensity across spectral channels:

$$\text{Brightness} = \frac{\text{Red} + \text{Green} + \text{Blue} + \text{NIR}}{4} \quad (2)$$

All spectral values were normalized by scaling pixel intensities to the range $[0, 1]$.

For CNN models, raw image patches were used directly without feature extraction. The four spectral bands were stacked as input channels, and all pixel values were normalized to the range $[0, 1]$.

2.2 Training Details

For Random Forest models, we used the scikit-learn implementation and trained both baseline and tuned configurations. Hyperparameters were selected through a small grid search, including the number of estimators, maximum tree depth, and minimum samples per leaf, to balance model performance and overfitting.

For SVM models, we evaluated multiple kernels, including linear, polynomial, and radial basis function (RBF). The RBF kernel was selected based on validation performance. The hyperparameters C and γ were tuned to optimize classification accuracy and class balance. All input features were standardized using `StandardScaler` prior to training.

For CNN models, we trained a convolutional neural network using PyTorch. The architecture consisted of three convolutional layers with ReLU activations, each followed by max pooling (MaxPool2d). These layers were followed by fully connected layers for classification. Dropout regularization was applied to reduce overfitting. We experimented with different learning rates and dropout rates, selecting the best-performing configuration based on validation accuracy. Models were trained for a fixed number of 10 epochs. All models were trained using an 80/20 stratified train-validation split.

2.3 Datasets and Tasks

The primary task in this study is binary image classification of satellite image patches into *cloud* and *no cloud* categories. The dataset consists of 5,155 filtered patches, which were used consistently across all experiments.

For the Random Forest and SVM models, we evaluated two input representations: (1) statistical features derived from spectral bands, and (2) an extended feature set including NDVI and brightness. These configurations allow us to assess the impact of feature engineering on model performance.

For the CNN model, raw multi-spectral image patches were used directly as input without feature extraction. This enables the model to learn spatial and spectral patterns directly from the data. Comparing CNN performance with feature-based models allows us to evaluate the trade-offs between learned representations and manually engineered features.

2.4 Metrics

Model performance was primarily evaluated using accuracy, which measures the proportion of correctly classified samples. Accuracy was used as the main metric for comparing model performance across experiments. To assess class-specific behavior, we additionally examined recall for the cloud and no-cloud classes.

To evaluate generalization, we performed 5-fold cross-validation on selected models. The mean and standard deviation of cross-validation accuracy were reported to assess model stability across different data splits. For CNN models, we also monitored training loss and both training and validation accuracy across epochs to evaluate convergence behavior and detect overfitting.

3 Results

3.1 Random Forests

3.1.1 Hyperparameter Tuning

We first trained a baseline Random Forest model (M1) using default parameters, achieving a validation accuracy of 93.01%. To improve generalization, we explored a range of hyperparameters, including the number of estimators, maximum tree depth, minimum samples per leaf, and class weighting. Selected configurations are shown in Table 1.

Increasing the minimum samples per leaf and constraining tree depth reduced overfitting, as reflected by smaller training-validation gaps, but slightly decreased validation accuracy. We performed 5-fold cross-validation on two candidate models: the highest-accuracy model (M2) and a lower-gap model (M3). Model M2 achieved a mean accuracy of 90.1% (std = 2.5%), while M3 achieved 88.5% (std = 2.9%), indicating that M2 performs better and generalizes more consistently. Based on these results, M2 was selected as the final Random Forest configuration.

Model	Trees	Depth	Leaf	Weight	Val Acc	Gap
M1	100	None	1	None	93.0%	0.070%
M2	300	None	5	None	92.6%	0.040%
M3	300	10	5	None	91.9%	0.029%
M4	300	10	1	None	92.5%	0.051%
M5	300	10	5	Balanced	91.4%	0.033%

Table 1: Random Forest hyperparameter tuning results.

3.1.2 Feature Engineering

We extended the feature set by incorporating NDVI and brightness, using the same Random Forest configuration for a controlled comparison. As shown in Table 2, the extended features slightly improved validation accuracy from 93.0% to 93.5% and increased the mean cross-validation accuracy from 90.1% to 90.7%.

Model	Val Acc	CV Acc	Top Feature	Second Feature
Original Features	93.0%	90.1%	NIR_max	blue_mean
Extended Features	93.5%	90.7%	NDVI_mean	blue_mean

Table 2: Random Forest performance and feature importance comparison.

3.2 SVMs

3.2.1 Hyperparameter Tuning

We first evaluated a baseline SVM using default parameters, achieving a validation accuracy of 88.8%. To determine an appropriate kernel, we compared linear, polynomial, and radial basis function (RBF) kernels, as shown in Table 3. The RBF kernel outperformed the linear and polynomial kernels, indicating that nonlinear decision boundaries are required to capture the structure of the data.

After selecting the RBF kernel, we tuned the regularization parameter C , which controls the tradeoff between maximizing the margin and minimizing classification error. Increasing C improved model performance, with $C = 10$ achieving a validation accuracy of 92.4% (Table 4).

We then tuned the kernel parameter γ , which controls the influence of individual training samples on the decision boundary. The best performance was achieved with $C = 10$ and $\gamma = 0.1$, resulting in a validation accuracy of 94.0% (Table 4). This configuration also demonstrated strong class-wise performance, with recall values of 0.91 for the no-cloud class and 0.96 for the cloud class.

We performed 5-fold cross-validation on the final SVM model. The model achieved a mean cross-validation accuracy of 91.2% with a standard deviation of 2.1%, indicating stable performance across different data splits.

Kernel	Parameters	Val Accuracy
Linear	Default	80.5%
Polynomial	Default	81.6%
RBF	Default	88.8%

Table 3: Comparison of SVM kernels.

Configuration	Parameters	Val Accuracy
Baseline (RBF)	Default	88.8%
Tuned C	$C = 10$	92.4%
Tuned $C + \gamma$	$C = 10, \gamma = 0.1$	94.0%

Table 4: SVM hyperparameter tuning results using RBF kernel.

3.2.2 Feature Engineering

We evaluated the impact of feature engineering by incorporating NDVI and brightness into the SVM model. As shown in Table 5, the extended feature set improved validation accuracy from 94% to 94.8%. The improvement is slight but could indicate that additional features provide useful information in this model. Class-wise recall remained strong across both configurations, with a slight improvement in the no-cloud class.

We performed 5-fold cross-validation on the extended feature model. The model achieved a mean accuracy of 92.1% with a standard deviation of 1.9%, indicating stable performance across data splits. Compared to the original feature model, both the mean accuracy increased and the variance decreased.

Feature Set	Val Acc	CV Acc	CV Std	Recall (No Cloud / Cloud)
Original Features	94.0%	91.2%	2.1%	0.91 / 0.96
Extended Features	94.8%	92.1%	1.9%	0.92 / 0.96

Table 5: SVM performance comparison with and without engineered features.

4 Deep CNNs

We evaluated a convolutional neural network trained directly on multi-spectral image patches. Unlike the Random Forest and SVM models, the CNN learns spatial and spectral patterns directly from the input data without engineered features.

A baseline model without dropout (learning rate = 0.001) achieved a validation accuracy of 88.6% and exhibited clear overfitting, with training accuracy increasing while validation performance degraded. Reducing the learning rate to 0.0001 improved performance but did not remain less stable compared to regularized models.

Introducing dropout significantly improved generalization. With a learning rate of 0.0001 and dropout of 0.5, the model achieved the best validation accuracy of 94.9% and more stable training behavior. While models without dropout could achieve competitive accuracy

at lower learning rates, they consistently showed greater overfitting compared to regularized models.

Configuration	Learning Rate	Dropout	Best Val Acc
Baseline CNN	0.001	0.3	89.6%
Lower LR	0.0001	0.3	93.0%
Higher Dropout	0.0001	0.5	94.9%
No Dropout	0.001	0.0	89.3%
No Dropout (Low LR)	0.0001	0.0	94.3%

Table 6: CNN training configurations and validation performance.

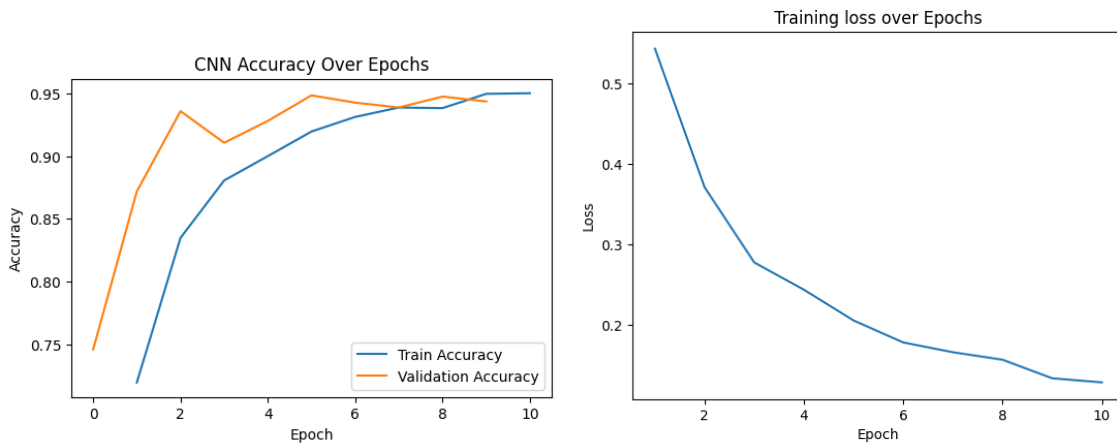


Figure 2: Training and validation accuracy (left) and training loss (right) for the CNN model.

As shown in Figure 2, reducing the learning rate improves training stability, while dropout reduces overfitting and leads to more consistent validation performance.

4.1 Model Comparison

Table 7 summarizes the best-performing configuration for each model. The CNN achieved the highest validation accuracy, followed closely by the SVM and Random Forest. Notably, the SVM performance differs from the CNN by only 0.001, indicating nearly identical performance between the two models.

Model	Best Configuration	Val Accuracy
Random Forest	300 trees, leaf=5 + NDVI/brightness	93.5%
SVM	RBF ($C = 10$, $\gamma = 0.1$) + NDVI/brightness	94.8%
CNN	LR=0.0001, dropout=0.5	94.9%

Table 7: Comparison of best-performing models.

5 Discussion

Across all models, we observe clear trade-offs between model complexity, interpretability, and generalization. While the CNN achieves the highest overall accuracy, classical models performed comparably to the deep CNN in this study. In particular, the SVM achieved a validation accuracy within 0.001 of the CNN, indicating nearly identical performance despite relying on engineered features.

Feature engineering had a significant impact on Random Forest performance. The addition of NDVI and brightness features led to slight improvements in both validation accuracy and cross-validation performance. Feature importance analysis showed that NDVI mean was the most influential feature, ranking above all original spectral features, including NIR and visible bands. This suggests that vegetation-related information plays a key role in distinguishing cloud-covered regions. The overall brightness feature contributed but to a lesser extent.

The SVM model demonstrated strong performance after tuning, particularly with the RBF kernel. The RBF kernel outperformed linear and polynomial kernels, indicating that nonlinear decision boundaries are necessary to separate cloud and no-cloud classes in the feature space. Increasing the regularization parameter C improved performance by allowing the model to better fit the training data, while tuning γ further refined the decision boundary. Although feature engineering improved SVM performance, the gains were less pronounced than for Random Forest, suggesting that SVMs are already effective at leveraging the original statistical features.

The CNN achieved the highest overall performance by learning spatial and spectral patterns directly from the raw image patches. However, its performance was highly sensitive to hyperparameter choices. A higher learning rate resulted in unstable training behavior, with validation performance fluctuating across epochs. Reducing the learning rate improved convergence and produced smoother training and validation curves. Dropout was critical for controlling overfitting: models without dropout achieved competitive training accuracy but showed larger gaps between training and validation performance. Increasing dropout to 0.5 reduced this gap and resulted in the most stable and best-performing model.

Although threshold sensitivity was explored during experimentation, it was not included in this report due to space constraints. The data processing pipeline was designed to support flexible threshold selection, and future work could investigate how different cloud coverage thresholds impact model performance and class balance. Additionally, extending the task to a multi-class setting, such as distinguishing between low, medium, and high cloud coverage, could provide more nuanced insights and better reflect real-world remote sensing applications.

6 Conclusion

In this study, we addressed the problem of patch-level cloud classification using multi-spectral satellite imagery from the 38-Cloud dataset. We evaluated three models to compare the effectiveness of feature-based and deep learning approaches.

Across all models, performance improved with hyperparameter tuning. For classical models, feature engineering played a key role in improving performance, with the domain-

related feature NDVI contributing to classification accuracy. The SVM model benefited from nonlinear decision boundaries through the RBF kernel, while the Random Forest model provided insight into feature importance and model behavior.

Overall, these results demonstrate that classical models can achieve competitive performance when combined with domain-informed feature engineering, while CNNs provide the strongest performance when properly tuned. This reflects a trade-off between interpretability and predictive power. Feature-based models, particularly Random Forests, allow us to identify important variables such as NDVI and brightness, providing insight into the underlying decision process. In contrast, CNNs capture more complex spatial and spectral relationships.

Future work could explore extending this approach to more complex classification settings, such as multi-class labeling of cloud coverage (e.g., low, medium, and high cloud presence). Additionally, while threshold sensitivity was explored during experimentation, further analysis could investigate how different definitions of cloud presence impact model performance in real-world applications.

References

- [1] J. A. Caraballo-Vega, M. L. Carroll, C. S. R. Neigh, and S. Anwar. Cloud masking using machine learning approaches. *Remote Sensing of Environment*, 284:113332, 2023.
- [2] Kaiqiang Ge, Jiayin Liu, Feng Wang, Bo Chen, and Yuxin Hu. A cloud detection method based on spectral and gradient features for sdsat-1 multispectral images. *Remote Sensing*, 15(1):24, 2023.
- [3] Michael D. King, Steven Platnick, W. Paul Menzel, Steven A. Ackerman, and Paul A. Hubanks. Spatial and temporal distribution of clouds observed by modis onboard the terra and aqua satellites. *IEEE Transactions on Geoscience and Remote Sensing*, 51(7):3826–3852, 2013.
- [4] S. Mohajerani, T. A. Krammer, and P. Saeedi. "A Cloud Detection Algorithm for Remote Sensing Images Using Fully Convolutional Neural Networks". In *2018 IEEE 20th International Workshop on Multimedia Signal Processing (MMSP)*, pages 1–5, Aug 2018.
- [5] S. Mohajerani and P. Saeedi. Cloud-net: An end-to-end cloud detection algorithm for landsat 8 imagery. In *IGARSS 2019 - 2019 IEEE International Geoscience and Remote Sensing Symposium*, pages 1029–1032, July 2019.
- [6] Michal Segal-Rozenhaimer, Alan Li, Kamalika Das, and Ved Chirayath. Cloud detection algorithm for multi-modal satellite imagery using convolutional neural-networks (cnn). *Remote Sensing of Environment*, 237:111446, 2020.
- [7] Wenbin Wu, Jing Luo, Xiaolin Hu, Hong Yang, and Yansheng Yang. A thin-cloud mask method for remote sensing images based on sparse dark pixel region detection. *Remote Sensing*, 10(4):617, 2018.